

so that the `Let` constructor takes a list of bindings, where a binding is a pair of a variable name and an expression. The example above would be represented as:

```
Let ([("x1", ...); ("x2", ...)], Prim("+", Var "x1", Var "x2"))
```

Revise the `eval` interpreter from `Intcomp1.fs` to work for the `expr` language extended with multiple sequential `let`-bindings.

Exercise 2.2 Revise the function `freevars : expr -> string list` to work for the language as extended in Exercise 2.1. Note that the example expression in the beginning of Exercise 2.1 has no free variables, but `let x1 = x1+7 in x1+8 end` has the free variable `x1`, because the variable `x1` is bound only in the body (`x1+8`), not in the right-hand side (`x1+7`), of its own binding. There *are* programming languages where a variable can be used in the right-hand side of its own binding, but ours is not such a language.

Exercise 2.3 Revise the `expr-to-texpr` compiler `tcomp : expr -> texpr` from `Intcomp1.fs` to work for the extended `expr` language. There is no need to modify the `texpr` language or the `teval` interpreter to accommodate multiple sequential `let`-bindings.

Exercise 2.4 Write a bytecode assembler (in F#) that translates a list of bytecode instructions for the simple stack machine in `Intcomp1.fs` into a list of integers. The integers should be the corresponding bytecodes for the interpreter in `Machine.java`. Thus you should write a function `assemble : sinstr list -> int list`.

Use this function together with `scomp` from `Intcomp1.fs` to make a compiler from the original expressions language `expr` to a list of bytecodes `int list`.

You may test the output of your compiler by typing in the numbers as an `int` array in the `Machine.java` interpreter. (Or you may solve Exercise 2.5 below to avoid this manual work).

Exercise 2.5 Modify the compiler from Exercise 2.4 to write the lists of integers to a file. An F# list `inss` of integers may be output to the file called `fname` using this function (found in `Intcomp1.fs`):

```
let intsToFile (inss : int list) (fname : string) =
    let text = String.concat " " (List.map string inss)
    System.IO.File.WriteAllText(fname, text)
```

Then modify the stack machine interpreter in `Machine.java` to read the sequence of integers from a text file, and execute it as a stack machine program. The name of the text file may be given as a command-line parameter to the Java program. Reading numbers from the text file may be done using the `StringTokenizer` class or `StreamTokenizer` class; see e.g. *Java Precisely* [4, Example 194].

It is essential that the compiler (in F#) and the interpreter (in Java) agree on the intermediate language: what integer represents what instruction.